DESENVOLVIMENTO BACK-END

Arquitetura MVC e Design de Código com SOLID

Unidade II · Organizando o código para que ele continue fácil de mudar

MVC · Controller · Service · Repository · SOLID · Refatoração · Injeção de dependência

🎯 Objetivos de aprendizagem

🧩

Compreender

o padrão MVC e por que, em APIs, a "visão" é o JSON

🔁

Explicar

o ciclo de vida de uma requisição no Laravel

🧱

Diferenciar

as camadas Controller, Service e Repository

📐

Relacionar

cada princípio SOLID ao problema que ele resolve

👃

Identificar

maus cheiros que pedem refatoração

💉

Analisar

como a injeção de dependência torna o código testável

😢 "É só mudar a média de 6 para 7..."

Um pedido simples da coordenação:

- a regra está **misturada** com SQL, HTML e e-mail
- o mesmo cálculo está **copiado** em vários arquivos
- cada correção gera um **novo defeito** em outro lugar
- **não há testes** para confirmar nada

🍽️

Analogia: a cozinha

Imagine uma cozinha em que **o mesmo cozinheiro** atende o salão, prepara o prato, lava a louça e faz as compras. Funciona com 3 mesas; com 50, vira **caos**.

Organizar o código não é questão de gosto: é questão de **custo**. Quanto mais confusa a estrutura, mais **caro** fica evoluir o sistema.

🧩 O padrão MVC

🗄️

Model

os **dados** e as **regras de negócio**, incluindo o acesso ao banco

🖼️

View

apresenta o resultado: uma página HTML ou um **JSON**

🎛️

Controller

recebe a requisição, aciona os models e escolhe a view

Requisição ➜ **Controller** ➜ **Model** ➜ **View**

💡 **Na secretaria acadêmica:** o atendente do balcão é o **Controller**; arquivos e normas são o **Model**; a declaração impressa entregue ao aluno é a **View**.

📤 Em APIs, a "visão" é o JSON

✅ Vantagens	⚠️ Limitações
separa dados, apresentação e controle	não diz onde ficam regras **complexas**
pastas e classes **previsíveis**	tende a **fat controller** ou **fat model**
várias Views para o **mesmo Model**	exige **disciplina** da equipe

📱

Mesmo Model, duas Views

Portal web: `view('cursos.index')` gera HTML.
API: `response()->json($cursos)` devolve dados.
No Laravel, a "visão" da API vira **API Resource** (Unidade III).

O MVC é um **ponto de partida**, não o ponto de chegada: sistemas maiores pedem **camadas adicionais**.

🔁 O ciclo de vida de uma requisição no Laravel



- A **resposta** volta **pelos middlewares** antes de sair
- Autenticação e logs de acesso pertencem aos **middlewares**, não ao controller
- **Spring:** `DispatcherServlet` + filters · **Express:** `app.use()` antes das rotas

> 💡 O controller é **apenas uma parte do caminho**: quando entende o ciclo em um ecossistema, você **reconhece** a mesma estrutura nos demais.

🧱 Controller → Service → Repository

🎛️ **Controller**
HTTP

→

🧠 **Service**
regras de negócio

→

🗄️ **Repository**
acesso a dados

Camada	Responsabilidade	Não deve	Analogia
Controller	traduzir HTTP em chamadas e respostas	conter **regras** ou **SQL**	atendente do balcão
Service	executar regras e **casos de uso**	conhecer `Request` ou SQL	responsável pelo setor
Repository	buscar e guardar **dados**	decidir regras de negócio	arquivista

💡 Cada camada conversa **apenas com a camada imediatamente abaixo**. O Service não sabe nada de HTTP; o Repository esconde se os dados vêm do banco, de uma API ou de um arquivo.

🔧 Antes e depois: o cadastro de alunos

```php
// ❌ ANTES: controller gordo
public function store(Request $request) {
  if (!$request->nome) { /* validação manual */ }
  $existe = DB::select('SELECT ...', [$email]); // SQL
  if ($existe) return response()->json(..., 409);
  $ra = date('Y') . random_int(0, 99999);  // regra
  DB::table('alunos')->insert([...]);  // persistência
  Mail::raw("Seu RA é {$ra}", ...);    // e-mail
  return response()->json(['ra' => $ra], 201);
}
```

```php
// ✅ DEPOIS: controller fino
public function __construct(
  private readonly CadastroAlunoService $cadastro
) {}                                    // injetado

public function store(Request $request) {
  $dados = $request->validate([...]); // 1. validar
  $aluno = $this->cadastro
                ->cadastrar($dados);  // 2. delegar
  return response()->json(            // 3. responder
    ['id' => $aluno->id, 'ra' => $aluno->ra], 201);
}
```

📌 **O que demonstra:** o controller passa a ter só **três passos**. A regra do e-mail duplicado vai para o **Service** (testável sem HTTP), o SQL vai para o **Repository** (trocável) e o envio de mensagens fica atrás de uma **interface**.

📐 SOLID: cinco princípios, um objetivo

🎯

S · Responsabilidade Única

um módulo responde a **um único ator**, ou seja, tem um só motivo para mudar

🔌

O · Aberto/Fechado

aberto para extensão, **fechado** para modificação

🔄

L · Substituição de Liskov

a subclasse **substitui** a superclasse sem quebrar nada

✂️

I · Segregação de Interfaces

ninguém depende de métodos que **não usa**

🔃

D · Inversão de Dependência

regras e detalhes dependem de **abstrações**

💰

Objetivo comum

reduzir o **custo de mudança**: uma regra nova altera **poucos arquivos**

🎯 SRP: um único motivo para mudar

❌

Sem SRP: RelatorioDesempenho

- calcula a média (coordenação)
- monta o HTML (comunicação)
- envia o e-mail (infraestrutura)

➜ **três motivos** para mudar; testar a média **envia e-mail**

✅

Com SRP: classes coesas

- `CalculadoraDesempenho`
- `FormatadorRelatorioHtml`
- `EnvioRelatorioDesempenho` coordena

➜ o cálculo é testado **isoladamente**, com `MEDIA_APROVACAO` explícita

"Um módulo deve ser responsável por **um, e apenas um, ator**" (Martin, 2019). A pergunta certa é: **quem pede as mudanças?**

🔌 OCP: novos descontos sem mexer no que funciona

```php
interface Desconto {                                    // a abstração
    public function aplicar(float $valor): float;
}
final class DescontoPercentual implements Desconto {   // cada desconto é uma classe
    public function __construct(private float $percentual) {}
    public function aplicar(float $valor): float {
        return round($valor * (1 - $this->percentual / 100), 2);
    }
}
final class CalculadoraMensalidade {                    // fechada para modificação
    public function calcular(float $valor, Desconto ...$descontos): float {
        foreach ($descontos as $d) { $valor = $d->aplicar($valor); } // aplica um a um
        return $valor;                                  // convênio 20% + pontualidade 5%: 1200 → 912
    }
}
```

📌 **O que demonstra: antes**, um `switch` com `'BOLSA_INTEGRAL'`, `'CONVENIO'`, `'PONTUALIDADE'`... crescia a cada desconto novo. **Agora**, o "desconto para irmãos" é **uma classe nova** que implementa `Desconto`, e os descontos ainda podem ser **combinados**.

🔄 LSP e ✂️ ISP: contratos confiáveis

🔄

Substituição de Liskov

❌ `NotificadorSms` lança exceção acima de **160 caracteres**: exige mais que a classe-mãe.

✅ divide a mensagem em partes e **cumpre a promessa** da superclasse.

🚩 Sinal: `if ($n instanceof NotificadorSms)`

✂️

Segregação de Interfaces

❌ `RepositorioAcademico` com listar, salvar, excluir, **exportarPdf**, **importarPlanilha**.

✅ `LeituraRepositorio`, `EscritaRepositorio`, `ExportadorPdf`.

🚩 Sinal: métodos **vazios** ou que lançam exceção

💡 Herança deve modelar **comportamento compatível**, não só uma relação "é um" do mundo real: o quadrado é um retângulo na matemática, mas não no código.

🔃 DIP: a regra não depende do detalhe

❌

Sem inversão

O `MatriculaService` faz `new MySqlMatriculaRepository()`

- migrar para PostgreSQL ➜ **alterar o service**
- testar sem banco ➜ **impossível**

✅

Com inversão

O service declara no construtor `MatriculaRepositoryInterface`

- a implementação é escolhida **fora dele**
- testes com repositórios **em memória**

Módulos de **alto nível** (regras) e de **baixo nível** (banco, e-mail) dependem de **abstrações**: o detalhe passa a depender da regra, e não o contrário.

👃 Maus cheiros e refatoração

Mau cheiro	Como aparece no back-end	Remédio
Função longa	`store()` com centenas de linhas	SRP · extrair função
Classe grande	controller ou service que faz "tudo"	SRP · extrair classe
Código duplicado	mesma regra de situação em vários controllers	extrair função
Switches repetidos	`if` / `switch` por tipo espalhado	OCP · polimorfismo
Lista longa de parâmetros	métodos com seis ou mais argumentos	DTO · objeto de parâmetro

Refatorar é mudar a estrutura interna **sem alterar o comportamento observável** (Fowler, 2019): em **pequenos passos**, rodando os **testes** a cada passo.

💉 Injeção de dependência e inversão de controle



- A classe **declara** o que precisa no **construtor**; alguém de fora **entrega pronto**
- O container monta a cadeia **recursivamente**; você só faz o `bind` da interface
- **Tempo de vida:** `bind` (novo a cada uso), `singleton` (um só), `scoped` (um por requisição)
- Nos testes, entra um **fake em memória** e o service **não muda nada**

💡 **DIP** é o princípio; **injeção de dependência** é a técnica; **IoC** é o framework, e não o seu código, controlar a criação dos objetos.

🍽️ Arquitetura no dia a dia

Conceito	No dia a dia	Na API acadêmica
Camadas	balcão, setor responsável e arquivo da secretaria	Controller, Service, Repository
SRP	o professor não emite boleto nem limpa a sala	média, layout e envio em classes separadas
OCP	tomada padrão: aparelho novo, **mesma tomada**	novo desconto = nova classe `Desconto`
DIP	o motorista depende da **habilitação**, não de um carro	service depende da **interface**
Injeção de dependência	o cirurgião **recebe** o bisturi pronto	o container entrega o repositório

💡 Se uma classe precisa de **dependências demais** no construtor, desconfie: ela provavelmente acumula responsabilidades e **viola o SRP**.

🌍 Aplicação em sistemas reais

🏚️

Sistema legado

controller de **1.500 linhas** refatorado em Repository, Service e classes de desconto, com testes a cada extração

💳

Pagamentos

novos meios de pagamento como **novas classes** de uma mesma interface (OCP)

📲

Notificações

trocar e-mail por WhatsApp mudando só o **bind** da `NotificadorInterface`

Protótipo tudo na rota → **Controller + Model** → **Surgiram regras?** Service → **Consulta reutilizada? Repository**

💡 **Comece simples.** Um service que só repassa `$repositorio->listar()` é burocracia: extraia camadas quando a **complexidade justificar**.

Síntese dos conceitos

1. **MVC** separa dados, apresentação e controle
2. Em **APIs**, a **View** é a representação em **JSON**
3. **Controller** (HTTP) ➜ **Service** (regras) ➜ **Repository** (dados)
4. **Middlewares** atuam antes e depois do controller

1. **SOLID** existe para reduzir o **custo de mudança**
2. **Maus cheiros** indicam onde refatorar
3. **Refatorar** = pequenos passos + **testes**
4. **Injeção de dependência** torna o código **testável**

Código bom não é o que **funciona hoje**: é o que continua **fácil de mudar amanhã**.

📝 Reflexão

🧩 Desafio: aplique o DIP

```php
class MatriculaService {
    public function __construct(
        private readonly ________ $matriculas, // interface ou classe concreta?
    ) {}
}

// app/Providers/AppServiceProvider.php
$this->app->____(                          // bind, singleton ou scoped?
    MatriculaRepositoryInterface::class,
    EloquentMatriculaRepository::class,
);
```

💬 Reflita

1. No seu último projeto, qual classe tinha **mais de um motivo** para mudar?
2. Quando criar um **Service** é burocracia, e não arquitetura?
3. Como testar a regra de **vagas esgotadas** sem um **banco de dados**?